

CITS5017 Deep Learning

Topic 4: Processing Sequences Using RNNs and CNNs

Associate Professor Du Huynh
(Unit Coordinator and Lecturer)
UWA

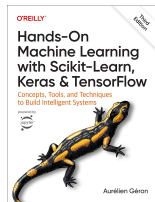
Semester 2, 2026



THE UNIVERSITY OF
WESTERN
AUSTRALIA

Chapters 15.

Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow



Processing Sequences Using RNNs and CNNs

Here are the main topics we will briefly cover:

- Various application areas of RNNs
- Recurrent neurons
- Memory cells
- Input and output sequences
- Basic RNNs in TensorFlow
- Handling variable length input and output sequences
- Training to predict time series
- Deep RNNs
- Difficulty of training over many time steps
- LSTM architecture

Recurrent Neural Networks and Various Applications

- *Recurrent neural networks* (RNNs) is a class of nets that can predict the future (well, up to a point, of course). They can analyze time series data such as stock prices, and tell you when to buy or sell. In autonomous driving systems, they can anticipate car trajectories and help avoid accidents. More generally, they can work on sequences of arbitrary lengths, rather than on fixed-sized inputs like all the nets we have discussed so far.
- As RNNs can take sentences, documents, or audio samples as input, they can be used in automatic translation, speech-to-text, *sentiment analysis*, or generating the next notes in a melody (e.g., see Google's *Magenta project* (<https://magenta.tensorflow.org/>)).

Recurrent Neurons

- So far we have mostly looked at feedforward neural networks, where the activations flow only in one direction, from the input layer to the output layer. A *recurrent neural network* (RNN) looks very much like a feedforward neural network, except it also has connections pointing backward.
- The simplest possible RNN, composed of just one neuron receiving inputs, producing an output, and sending that output back to itself, as shown in the left diagram.

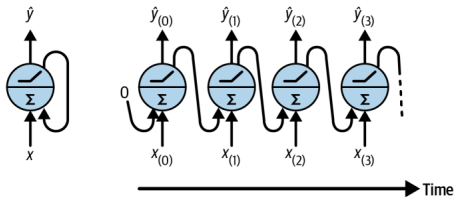


Figure 15-1. A recurrent neuron (left), unrolled through time (right)

- At each time step t (also called a *frame*), this recurrent neuron may receive the inputs $\mathbf{x}_{(t)}$ and its own output from the previous time step, $\hat{\mathbf{y}}_{(t-1)}$. This is called *unrolling the network through time* (right diagram).

Recurrent Neurons

A layer of recurrent neurons can be easily created. At each time step t , every neuron receives both the input vector $\mathbf{x}_{(t)}$ and the output vector from the previous time step $\mathbf{y}_{(t-1)}$, as shown below. Note that both the inputs and outputs are vectors now (when there was just a single neuron, the output was a scalar).

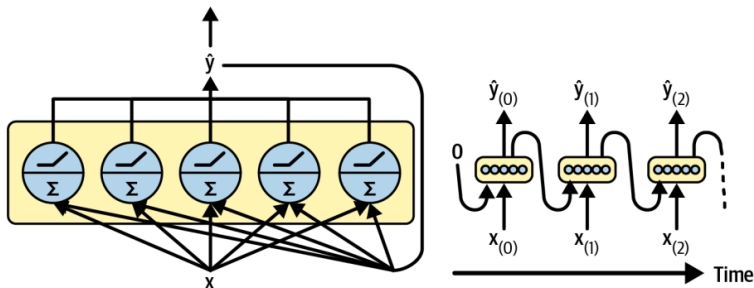


Figure 15-2. A layer of recurrent neurons (left), unrolled through time (right)

Memory Cells

- Notice that $\mathbf{y}_{(2)}$ depends on $\mathbf{x}_{(2)}$ and $\mathbf{y}_{(1)}$, which in turn depends on $\mathbf{x}_{(1)}$ and $\mathbf{y}_{(0)}$, and so on. So $\mathbf{y}_{(t)}$ is a function of all the inputs since time $t = 0$, i.e., $\mathbf{x}_{(0)}$, $\mathbf{x}_{(1)}$, $\dots$, $\mathbf{x}_{(t)}$.
- Since the output of a recurrent neuron at time step t is a function of all the inputs from previous time steps, you could say it has a form of *memory*. A part of a neural network that preserves some state across time steps is called a *memory cell* (or simply a *cell*).
- A single recurrent neuron, or a layer of recurrent neurons, is a very basic cell. There are more complex and powerful types of cells in more complex networks.

Memory Cells

We use $\mathbf{h}_{(t)}$ (where “h” stands for “hidden”) to denote a cell's **state** at time step t . In general, a cell's state at time step t is a function of some inputs at t and its state at $t-1$, i.e., $\mathbf{h}_{(t)} = f(\mathbf{x}_{(t)}, \mathbf{h}_{(t-1)})$. Its output $\hat{\mathbf{y}}_{(t)}$ is also a function of the previous state and the current inputs.

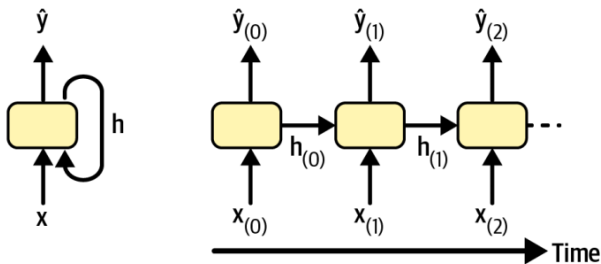


Figure 15-3. A cell's hidden state and its output may be different

Memory Cells

Mathematically, we have

$$\begin{aligned}\mathbf{h}_{(t)}^\top &= \varphi_h \left(\mathbf{h}_{(t-1)}^\top \mathbf{W}_h + \mathbf{x}_{(t)}^\top \mathbf{W}_x + \mathbf{b}_h^\top \right) \\ \mathbf{y}_{(t)}^\top &= \varphi_y \left(\mathbf{h}_{(t)}^\top \mathbf{W}_y + \mathbf{b}_y^\top \right)\end{aligned}$$

where

$\mathbf{x}_{(t)}^\top$ is an input instance (an n_{inputs} -dimensional row vector) at time step t ;

$\mathbf{y}_{(t)}^\top$ is the corresponding output instance (an n_{outputs} -dimensional row vector);

$\mathbf{h}_{(t)}^\top$ is the hidden state vector (an n_{neurons} -dimensional row vector) at time step t ;

$\mathbf{W}_h$ ($n_{\text{neurons}} \times n_{\text{neurons}}$), $\mathbf{W}_x$ ($n_{\text{inputs}} \times n_{\text{neurons}}$), and $\mathbf{W}_y$ ($n_{\text{neurons}} \times n_{\text{outputs}}$) are unknown weight matrices that need to be learned from the training process;

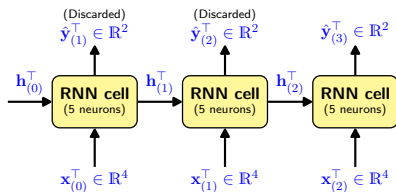
$\mathbf{b}_h^\top$ (n_{neurons} -dimensional row vector) and $\mathbf{b}_y^\top$ (n_{outputs} -dimensional row vector) are bias vectors that need to be learned also;

φ_h is an appropriate activation function (e.g., ReLU, tanh);

φ_y is an appropriate activation function (e.g., sigmoid for binary classification, or softmax combined with cross-entropy for multi-class classification)

Memory Cells (cont.)

For example, the inputs could be the *temperatures*, *precipitations*, *atmospheric pressures*, and *wind speeds* for three consecutive days (so $\mathbf{x}_{(0)}, \mathbf{x}_{(1)}, \mathbf{x}_{(2)} \in \mathbb{R}^4$) and we want to train a simple RNN with 5 neurons to predict the *temperature* and *precipitation* for the following day (i.e., predict $\hat{\mathbf{y}}_{(3)} \in \mathbb{R}^2$).



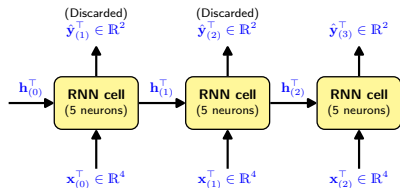
Then

$$\begin{aligned} \mathbf{W}_h &\in \mathbb{R}^{5 \times 5} \text{ and } \mathbf{h}_{(t)} \in \mathbb{R}^5, \\ \mathbf{W}_x &\in \mathbb{R}^{4 \times 5}, \\ \mathbf{W}_y &\in \mathbb{R}^{5 \times 2}. \end{aligned}$$

We are interested only in $\hat{\mathbf{y}}_{(3)}$, which will be compared with the ground truth $\mathbf{y}_{(3)}$ in the loss function (e.g., average MSE).

The initial hidden state vector $\mathbf{h}_{(0)}^T$ for each training instance is usually set to a zero vector.

Memory Cells (cont.)



Let m be our minibatch size. Then our training data $\mathbf{X}_{(t)} \in \mathbb{R}^{m \times 4}$ and $\mathbf{Y}_{(t)} \in \mathbb{R}^{m \times 2}$, for all t , and the equations on slide 9 become

$$\mathbb{R}^{m \times 5} \ni \mathbf{H}_{(t)} = \varphi_h (\mathbf{H}_{(t-1)} \mathbf{W}_h + \mathbf{X}_{(t)} \mathbf{W}_x + \mathbf{B}_h)$$

$$\mathbb{R}^{m \times 2} \ni \hat{\mathbf{Y}}_{(t)} = \varphi_y (\mathbf{H}_{(t)} \mathbf{W}_y + \mathbf{B}_y)$$

where

$$\mathbf{B}_h = \begin{bmatrix} \mathbf{b}_h^\top \\ \vdots \\ \mathbf{b}_h^\top \end{bmatrix} \in \mathbb{R}^{m \times 5} \quad \text{and} \quad \mathbf{B}_y = \begin{bmatrix} \mathbf{b}_y^\top \\ \vdots \\ \mathbf{b}_y^\top \end{bmatrix} \in \mathbb{R}^{m \times 2}$$

The training data are usually set up as 3D tensors: $\mathbf{X} \in \mathbb{R}^{N \times 3 \times 4}$ and $\mathbf{Y} \in \mathbb{R}^{N \times 2}$ for this example, where N is the training set size.

Input and Output Sequences

- 1 An RNN can take a sequence of inputs and produce a sequence of outputs, e.g., take N days of stock prices as input and produce the stock prices of $N - 1$ ago and of tomorrow.
- 2 An RNN can take a sequence of inputs and produce a single output at the last step, e.g., take in a sequence of words from a movie review and output a sentiment score (0 (bad) ... 4 (good))
- 3 An RNN can take a single input at $t = 0$ (and zeros for other time steps) and produce a sequence of outputs, e.g., take in an image and produce a sequence of words that could be a caption of that image.
- 4 Lastly, you could have a sequence-to-vector network, called an *encoder*, followed by a vector-to-sequence network, called a *decoder*. This two-step model is called an *encoder-decoder*. e.g., translating a sentence from one language to another language.

Recall that an *input* is just a vector (or a number) and an *output* is another vector (or number).

Input and Output Sequences

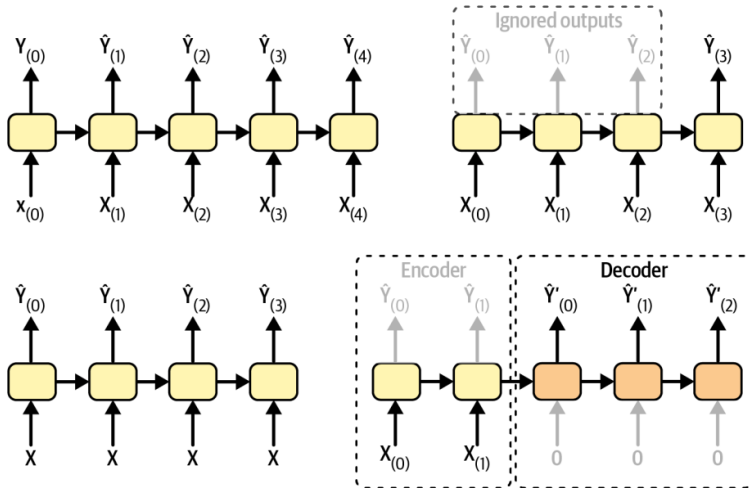


Figure 15-4. Sequence-to-sequence (top left), sequence-to-vector (top right), vector-to-sequence (bottom left), and encoder-decoder (bottom right) networks

Training RNNs

To train an RNN, the trick is to unroll it through time (like we just did) and then use regular backpropagation (Figure 15-5). This strategy is called *backpropagation through time* (BPTT).

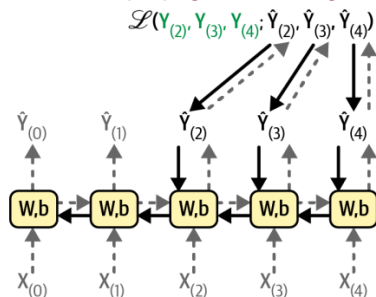


Figure 15-5. Backpropagation through time

In the example above, the loss function is computed based on the last three outputs only. The gradients of the loss function are then propagated backward through the unrolled network (solid arrows).

$\{\mathbf{X}_{(t)} \mid t = 0, \dots, 4\}$ denotes the batch of training sequences fed into the network;
 $\{\hat{\mathbf{Y}}_{(t)} \mid t = 0, \dots, 4\}$ denotes the corresponding computed output.

Forecasting a Time Series

```
tf.keras.utils.get_file(
    "ridership.tgz", "https://github.com/ageron/data/raw/main/ridership.tgz",
    cache_dir=".", extract=True)

import pandas as pd
from pathlib import Path

path = Path("datasets/ridership/CTA_-_Ridership_-_Daily_Boarding_Totals.csv")
df = pd.read_csv(path, parse_dates=["service_date"])
df.columns = ["date", "day_type", "bus", "rail", "total"] # shorter names
df = df.sort_values("date").set_index("date")
df = df.drop("total", axis=1) # no need for total, it's just bus + rail
df = df.drop_duplicates() # remove duplicated months (2011-10 and 2014-07)

>>> df.head()
```

	day_type	bus	rail
date			
2001-01-01	U	297192	126455
2001-01-02	W	780827	501952
2001-01-03	W	824923	536432
2001-01-04	W	870021	550011
2001-01-05	W	890426	557917

Note that data frame **df** has 3 columns: **day_type**, **bus**, and **rail**. Each row of **df** has the name that corresponds to the date the data was collected.

Forecasting a Time Series (cont.)

```
import matplotlib.pyplot as plt

df["2019-03":"2019-05"].plot(grid=True, marker=".", figsize=(8, 3.5))
save_fig("daily_ridership_plot") # extra code { saves the figure for the book
plt.show()
```

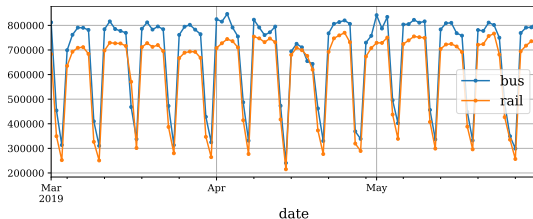


Figure 15-6. Daily ridership in Chicago

A similar pattern can be seen repeated each week. This is called a weekly *seasonality*. If we simply copy a past value to make our forecast, then it is called *naive forecasting*. **Naive forecasting is often a good baseline, and it can even be tricky to beat in some cases.**

Forecasting a Time Series (cont.)

In our example, copying the value from the previous week works better, due to the strong weekly seasonality.

```
diff_7 = df[["bus", "rail"]].diff(7)["2019-03":"2019-05"]

fig, axs = plt.subplots(2, 1, sharex=True, figsize=(8, 5))
df.plot(ax=axs[0], legend=False, marker=".") # original time series
df.shift(7).plot(ax=axs[0], grid=True, legend=False, linestyle=":") # lagged
diff_7.plot(ax=axs[1], grid=True, marker=".") # 7-day difference time series
axs[0].set_ylim([170_000, 900_000]) # extra code { beautifies the plot
save_fig("differencing_plot") # extra code { saves the figure for the book
plt.show()
```

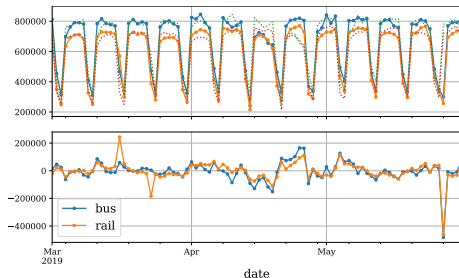


Figure 15-7. Time series overlaid with 7-day lagged time series (top), and difference between t and $t - 7$ (bottom)

Forecasting a Time Series (cont.)

The *mean absolute error* (MAE), also called the *mean absolute deviation* (MAD) is:

```
>>> diff_7.abs().mean()  
bus      43915.608696  
rail     42143.271739  
dtype: float64
```

Dividing the forecast errors above by the target values gives us the *mean absolute percentage error* (MAPE):

```
>>> targets = df[["bus", "rail"]]["2019-03":"2019-05"]  
>>> (diff_7 / targets).abs().mean()  
bus      0.082938  
rail     0.089948  
dtype: float64
```

While the prediction for number of bus riders using 7-days' shift has a higher MAE than that for the rail rider predictions, its MAPE is actually lower.

This *7-days' shift* prediction serves as a baseline for benchmarking our RNN methods later on.

Preparing the Data for Machine Learning Models

Data preparation is an important step in sequence processing as the input data usually does not come in the form that can be directly used.

Consider the following simple example:

```
my_series = [0, 1, 2, 3, 4, 5]
my_dataset = tf.keras.utils.timeseries_dataset_from_array(
    my_series,
    targets=my_series[3:], # the targets are 3 steps into the future
    sequence_length=3,
    batch_size=2
)
list(my_dataset)

# the output is:
[(<tf.Tensor: shape=(2, 3), dtype=int32, numpy=
  array([[0, 1, 2],
        [1, 2, 3]], dtype=int32)>,
  <tf.Tensor: shape=(2,), dtype=int32, numpy=array([3, 4], dtype=int32)>),
 (<tf.Tensor: shape=(1, 3), dtype=int32, numpy=array([[2, 3, 4]], dtype=int32)
  <tf.Tensor: shape=(1,), dtype=int32, numpy=array([5], dtype=int32)>)]
```

Preparing the Data for Machine Learning Models (cont.)

Our first step is to split our data into a training period, a validation period, and a test period. We will focus on the rail ridership for now.

```
rail_train = df["rail"]["2016-01":"2018-12"] / 1e6
rail_valid = df["rail"]["2019-01":"2019-05"] / 1e6
rail_test = df["rail"]["2019-06":] / 1e6

seq_length = 56
train_ds = tf.keras.utils.timeseries_dataset_from_array(
    rail_train.to_numpy(),
    targets=rail_train[seq_length:],
    sequence_length=seq_length,
    batch_size=32,
    shuffle=True,
    seed=42)

valid_ds = tf.keras.utils.timeseries_dataset_from_array(
    rail_valid.to_numpy(),
    targets=rail_valid[seq_length:],
    sequence_length=seq_length,
    batch_size=32)
```

Preparing the Data for Machine Learning Models (cont.)

Inspect 2 minibatches in `train_ds`:

```
index = 0
for xx,yy in train_ds.take(2):
    print(f"Minibatch {index}:")
    print("xx.shape =", xx.shape)
    print("yy.shape =", yy.shape)
    index += 1
```

The output is:

Minibatch 0:

xx.shape = (32, 56)

yy.shape = (32,)

Minibatch 1:

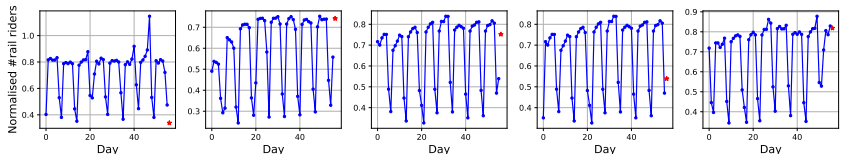
xx.shape = (32, 56)

yy.shape = (32,)

Preparing the Data for Machine Learning Models (cont.)

Visualise the training data:

```
_, axes = plt.subplots(1,5,figsize=(15,3), sharey=True)
history_timesteps = range(56)          # input: rail data for days 0 to 55
next_timestep = 56                     # predict: rail data for day 56
for xx,yy in train_ds.take(1):         # look at the first 5 sequences
    for ax,i in zip(axes, range(5)):    # in a minibatch
        ax.plot(history_timesteps, xx[i,:], "b.-")
        ax.plot(next_timestep, yy[i], "r*")
        ax.grid("on")
        ax.set_xlabel("Day")
        if i == 0:
            ax.set_ylabel("Normalised #rail riders")
plt.show()
```



Forecasting a Time Series

Forecasting Using a Linear Model

```
tf.random.set_seed(42)
model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(shape=[seq_length]), # seq_length = 56
    tf.keras.layers.Dense(1)
])

early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    monitor="val_mae", patience=50, restore_best_weights=True)
opt = tf.keras.optimizers.SGD(learning_rate=0.02, momentum=0.9)
model.compile(loss=tf.keras.losses.Huber(), optimizer=opt,
              metrics=["mae"])

history = model.fit(train_ds, validation_data=valid_ds, epochs=500,
                    callbacks=[early_stopping_cb])
```

This model has 57 trainable parameters (0 non-trainable) and it reaches a validation MAE of about 38,245 (your mileage may vary). Recall that the MAE using 7-days' shift (slide 18) is 42,143.

Forecasting a Time Series (cont.)

Forecasting Using a Simple RNN

```
tf.random.set_seed(42) # extra code - ensures reproducibility
model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(shape=[None, 1]),
    tf.keras.layers.SimpleRNN(1)
])
```

Note that all recurrent layers in Keras expect 3D inputs of shape `[batch size, time steps, dimensionality]`, where `dimensionality` is `1` for univariate time series and more for multivariate time series.

```
# define a utility function for training and evaluation
def fit_and_evaluate(model, train_set, valid_set, learning_rate, epochs=500):
    early_stopping_cb = tf.keras.callbacks.EarlyStopping(
        monitor="val_mae", patience=50, restore_best_weights=True)
    opt = tf.keras.optimizers.SGD(learning_rate=learning_rate, momentum=0.9)
    model.compile(loss=tf.keras.losses.Huber(), optimizer=opt, metrics=["mae"])
    history = model.fit(train_set, validation_data=valid_set, epochs=epochs,
                        callbacks=[early_stopping_cb])
    valid_loss, valid_mae = model.evaluate(valid_set)
    return valid_mae * 1e6
```

The validation MAE from this model is greater than **100,000**!

Forecasting a Time Series (cont.)

Forecasting Using a Simple RNN (cont.)

The large validation MAE was expected for two reasons:

- 1 The model only has a single recurrent neuron, so the only data it can use to make a prediction at each time step is the input value at the current time step and the output value from the previous time step. In other words, the RNN's memory is extremely limited: the whole model only has 3 parameters!

```
>>> model.summary()
Model: "sequential"
```

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 1)	3

```
=====
Total params: 3
Trainable params: 3
Non-trainable params: 0
```

Forecasting a Time Series (cont.)

Forecasting Using a Simple RNN (cont.)

The large validation MAE was expected for two reasons:

- 2 The time series contains values from 0 to about 1.4, but since the default activation function is `tanh`, the recurrent layer can only output values between -1 and $+1$. There's no way it can predict values between 1.0 and 1.4.

To fix these two issues, we will create a model (the `univar_model` on the next slide) with a larger recurrent layer, say 32 recurrent neurons, and we will add a dense output layer with a single output neuron and no activation function.

Compiling, fitting, and evaluating this `univar_model` as before, we should get a validation MAE around 27,703.

Forecasting a Time Series (cont.)

Forecasting Using a Simple RNN (cont.)

```
univar_model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(shape=[None, 1]),
    tf.keras.layers.SimpleRNN(32),
    tf.keras.layers.Dense(1) # no activation function by default
])
```

```
>>> univar_model.summary()
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
simple_rnn_1 (SimpleRNN)	(None, 32)	1088
dense (Dense)	(None, 1)	33

```
Total params: 1,121
```

```
Trainable params: 1,121
```

```
Non-trainable params: 0
```

Forecasting a Time Series (cont.)

Forecasting Using a Deep RNN

It is quite common to stack multiple layers of cells to form a *deep RNN*.

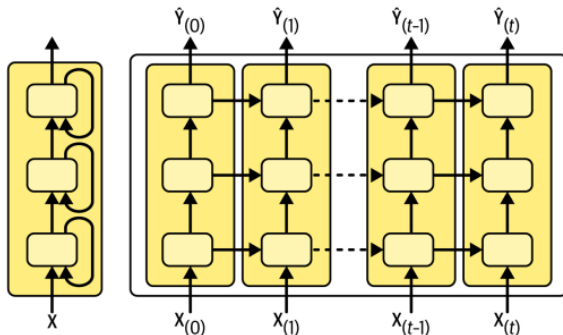


Figure 15-10. Deep RNN (left), unrolled through time (right)

Forecasting a Time Series (cont.)

Forecasting Using a Deep RNN (cont.)

Implementing a deep RNN with Keras is straightforward: just stack the recurrent layers, as shown below. **Important:** set `return_sequences=True` for all the `SimpleRNN` layers except for the last one.

```
deep_model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(shape=[None, 1]),
    tf.keras.layers.SimpleRNN(32, return_sequences=True), # 1,088 params
    tf.keras.layers.SimpleRNN(32, return_sequences=True), # 2,080 params
    tf.keras.layers.SimpleRNN(32), # 2,080 params
    tf.keras.layers.Dense(1) # 33 params
])
```

The above model has 5,281 trainable parameters (0 non-trainable parameters). Its validation MAE is about 29,672, which is worse than the previous “shallower” RNN. It looks like this RNN is a bit too large for our task.

Forecasting Multivariate Time Series

A great quality of neural networks is their flexibility: in particular, they can deal with multivariate time series with almost no change to their architecture. For example, we can try to predict the rail time series with *day type* as an additional input.

```
df_mulvar = df[["bus","rail"]]/1e6 # use both bus and rail series
df_mulvar["next_day_type"] = df["day_type"].shift(-1) # and tomorrow's type
df_mulvar = pd.get_dummies(df_mulvar) # one-hot encode the day type
mulvar_train = df_mulvar["2016-01":"2018-12"]
mulvar_valid = df_mulvar["2019-01":"2019-05"]
mulvar_test = df_mulvar["2019-06":]
mulvar_train.head()
```

	bus	rail	next_day_type_A	next_day_type_U	next_day_type_W
date					
2016-01-01	0.303321	0.319835	1	0	0
2016-01-02	0.448859	0.365509	0	1	0
2016-01-03	0.340540	0.287661	0	0	1
2016-01-04	0.829429	0.703185	0	0	1
2016-01-05	0.846789	0.727716	0	0	1

Forecasting Multivariate Time Series (cont.)

```
train_mulvar_ds = tf.keras.utils.timeseries_dataset_from_array(  
    tf.convert_to_tensor(mulvar_train, dtype=tf.float32), # use all 5 columns  
    targets=mulvar_train["rail"][seq_length:], # forecast only the rail series  
    [...], # the other 4 arguments are the same as earlier  
)  
valid_mulvar_ds = tf.keras.utils.timeseries_dataset_from_array(  
    tf.convert_to_tensor(mulvar_valid, dtype=tf.float32),  
    targets=mulvar_valid["rail"][seq_length:],  
    [...], # the other 2 arguments are the same as earlier  
)
```

Finally we create a simple RNN with 32 neurons and evaluate it using the `fit_and_evaluate` function (Slide 24):

```
mulvar_model = tf.keras.Sequential([  
    tf.keras.layers.InputLayer(shape=[None, 5]),  
    tf.keras.layers.SimpleRNN(32),  
    tf.keras.layers.Dense(1)  
])
```

This model reaches a validation MAE of 22,062 !!!

Forecasting Multiple Outputs

With small changes to the training and validation datasets, we can train a **multitask** RNN model to predict both the daily numbers of rail and bus riders.

```
train_multitask_ds = tf.keras.utils.timeseries_dataset_from_array(
    tf.convert_to_tensor(mulvar_train, dtype=tf.float32),
    targets=mulvar_train[["bus", "rail"]][seq_length:], # 2 targets per day
    [...])      # the other 4 arguments are the same as earlier
valid_multitask_ds = tf.keras.utils.timeseries_dataset_from_array(
    tf.convert_to_tensor(mulvar_valid, dtype=tf.float32),
    targets=mulvar_valid[["bus", "rail"]][seq_length:],
    [...])      # the other 2 arguments are the same as earlier

multitask_model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(shape=[None, 5]),
    tf.keras.layers.SimpleRNN(32),
    tf.keras.layers.Dense(2) ])

fit_and_evaluate(multitask_model, train_multitask_ds, valid_multitask_ds,
                  learning_rate=0.02)
```

Forecasting Multiple Outputs

This `multask_model` gives 27,855 MAE. Using the `predict` function to get the raw daily predictions:

```
Y_preds_valid = multask_model.predict(valid_multask_ds)
for idx, name in enumerate(["bus", "rail"]):
    mae = 1e6 * tf.keras.metrics.MeanAbsoluteError()(
        mulvar_valid[name][seq_length:].to_numpy(), Y_preds_valid[:, idx])
    print(name, int(mae))
```

we get the following MAEs for **bus** and for **rail**.

```
bus 26,812
rail 28,896
```

Different runs can yield slightly different results. In my 2024 run, the `multask_model` gave a total validation MAE of 24,421, and a validation MAE of 25,162 for **bus** and a validation MAE of 23,680 for **rail**.

Forecasting Several Time Steps Ahead

We could also forecast several time steps ahead by using the predicted value at time step $t - 1$ as the last input value for time step t : (Variable `rail_valid` below was defined on slide 20)

```
import numpy as np
# recall that rail_valid starts in 2019-01-01
X = rail_valid.to_numpy()[np.newaxis, :seq_length, np.newaxis] # one instance
for step_ahead in range(14):
    y_pred_one = univar_model.predict(X)
    X = np.concatenate([X, y_pred_one.reshape(1, 1, 1)], axis=1)
```

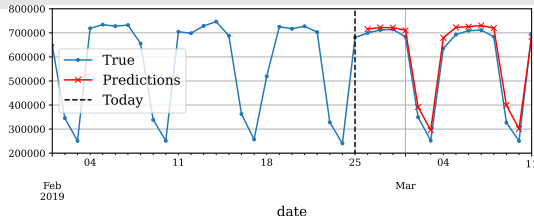


Figure 15-11. Forecasting 14 steps ahead, 1 step at a time.

Validation MAE = 30,734. **NOTE:** This method works only when the input and output vectors at the same features.

NOTE:
`univar_model`
(see slide 27)
was trained to
predict **one**
step ahead only.
(This diagram
may not look
the same as the
one in the
textbook.)

Forecasting Several Time Steps Ahead (cont.)

We can also train an RNN model to forecast the numbers of rail riders for 14 days ahead at the last training time step. We need to modify our code slightly to get the correct training and validation datasets:

```
def split_inputs_and_targets(mulvar_series, ahead=14, target_col=1):  
    return mulvar_series[:, :-ahead], mulvar_series[:, -ahead:, target_col]  
  
ahead_train_ds = tf.keras.utils.timeseries_dataset_from_array(  
    tf.convert_to_tensor(mulvar_train, dtype=tf.float32),  
    targets=None,  
    sequence_length=seq_length + 14,  
    batch_size=32,  
    shuffle=True,  
    seed=42  
)  
.map(split_inputs_and_targets)  
  
ahead_valid_ds = tf.keras.utils.timeseries_dataset_from_array(  
    tf.convert_to_tensor(mulvar_valid, dtype=tf.float32),  
    targets=None,  
    sequence_length=seq_length + 14,  
    batch_size=32  
)  
.map(split_inputs_and_targets)
```

Forecasting Several Time Steps Ahead (cont.)

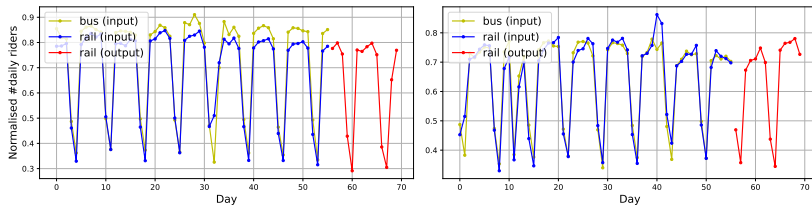
```
>>> print("Number of batches in ahead_train_ds =", len(ahead_train_ds))
>>> for x, y in ahead_train_ds.take(1): # inspect a minibatch
>>>     print(x.shape)
>>>     print(y.shape)
Number of batches in ahead_train_ds = 33
(32, 56, 5)
(32, 14)
```

Some simple data visualisation would help us confirm that our datasets are correctly set up.

```
_, axes = plt.subplots(1, 2, figsize=(15,4))
days_in = range(seq_length) # days 0 to 55
days_out = range(seq_length, seq_length+14) # days 56 to 69
for x, y in ahead_train_ds.take(1):
    for ax, id in zip(axes, range(2)):
        ax.plot(days_in, x[id, :, 0], "y.-", label="bus (input)")
        ax.plot(days_in, x[id, :, 1], "b.-", label="rail (input)")
        ax.plot(days_out, y[id], "r.-", label="rail (output)")
        ax.grid("on")
        ax.set_xlabel("Day")
        if id == 0:
            ax.set_ylabel("Normalised #daily riders")
        ax.legend(loc="upper left")
plt.show()
```

Forecasting Several Time Steps Ahead (cont.)

Visualisation of the first two sequences in a minibatch of training set `ahead_train_ds`. Each input timestep is a 5-dimensional vector. Only the number of rail riders is to be predicted per time step for 14 time steps ahead.

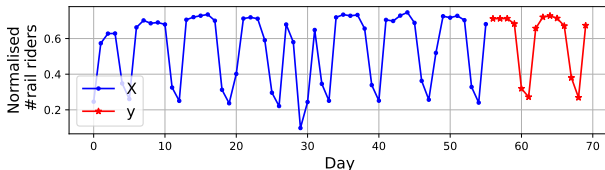


Forecasting Several Time Steps Ahead (cont.)

Next, we create an RNN, train, and evaluate it:

```
ahead_model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(shape=[None, 5]),
    tf.keras.layers.SimpleRNN(32),
    tf.keras.layers.Dense(14) ])
# Training code omitted...
X = mulvar_valid.to_numpy()[np.newaxis, :seq_length] # shape [1, 56, 5]
Y_pred = ahead_model.predict(X) # shape [1, 14]
```

This `ahead_model` gave a validation MAE 33,887 for rail.



This approach works quite well. Forecasting for just one day is obviously better than forecasting for 14 days into the future. However, even with 5 input features, `ahead_model` performed slightly worse than the `univar_model` which uses 1 feature and predicts 1 day at a time through looping.

Forecasting Using a Sequence-to-Sequence Model

Instead of training the model to forecast the next 14 values only at the very last time step, we can train it to forecast the predictions at each and every time step, e.g., use the training data from day 0 to day 9 to forecast the output from days 10 to day 19; then use the training data from day 1 to day 10 to forecast the output from day 11 to day 20, and so on. This can be done using the **Encoder-Decoder** architecture (See bottom-right diagram in Figure 15-4 (slide 13))

We will study the Encoder-Decoder architecture for processing sequences in **lecture topic 8** (*Transformers*) later.

Handling Long Sequences

- Just like the deep nets that we have seen in previous chapters, training an RNN on long sequences (i.e., many time steps) has the vanishing/exploding gradient problem and takes a long time to train or the training may be unstable. Moreover, when an RNN processes a long sequence, it will gradually forget the first inputs in the sequence.
- To fight the unstable gradients problem in RNNs, tricks, such as good parameter initialization, faster optimizers, and dropout should all be considered. However, nonsaturating activation functions (e.g., **ReLU**) may not help as much here, because the same weights are used in every time step and may lead to exploding outputs.
- You can reduce the risk above by using a smaller learning rate or a saturating activation function like the *hyperbolic tangent* (that's why it's the default).

Handling Long Sequences (cont.)

- *Batch normalisation* can be put over RNN layers but was reported to be not yielding good results when BN was used at each time step.
- *Layer normalization*¹ has been reported to be better for RNNs. It is very similar to batch normalization, but instead of normalizing across the batch dimension, layer normalization normalizes across the features dimension.

Example:

```
model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(shape=[None,5]),
    tf.keras.layers.SimpleRNNCell(32),
    tf.keras.layers.LayerNormalization(),
    tf.keras.layers.Dense(1)
])
```

¹ Jimmy Lei Ba et al., "Layer Normalization", arXiv preprint arXiv:1607.06450 (2016).

Due to the transformations that the data goes through when traversing an RNN, some information is lost at each time step. After a while, the RNN's state contains virtually no trace of the first inputs. To tackle this problem, various types of cells with long-term memory have been introduced.

The long short-term memory (LSTM) cell was proposed in 1997². If you consider the *Long Short-Term Memory* (LSTM) cell as a black box, it can be used very much like a basic cell, except it will perform much better:

- training will converge faster; and
- it will detect long-term dependencies in the data.

²Sepp Hochreiter and Jürgen Schmidhuber, "Long Short-Term Memory", Neural Computation 9, no. 8 (1997): 1735–1780.

LSTM Cell (cont.)

In TensorFlow, you can simply use a **LSTM** layer instead of a **SimpleRNN**:

```
# Number of trainable parameters: 685
lstm_model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(shape=[None, 1]),
    tf.keras.layers.LSTM(12),
    tf.keras.layers.Dense(1)
])
```

The above model gave a validation MAE of 30,262.

LSTM cells manage two state vectors: $\mathbf{h}_{(t)}$ and $\mathbf{c}_{(t)}$ (“c” stands for “cell”). You can think of $\mathbf{h}_{(t)}$ as the short-term state and $\mathbf{c}_{(t)}$ as the long-term state.

The key idea is that the network can learn what to store in the long-term state, what to throw away, and what to read from it.

LSTM Cell (cont.)

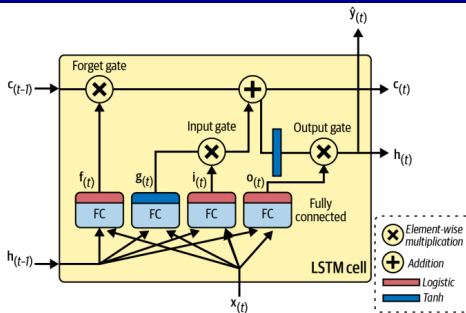


Figure 15-12. An LSTM cell

- As the long-term state $c_{(t-1)}$ traverses the network from left to right, it first goes through a *forget gate*, dropping some memories, and then it adds some new memories via the addition operation (which adds the memories that were selected by an *input gate*). The result $c_{(t)}$ is sent straight out, without any further transformation.

LSTM Cell (cont.)

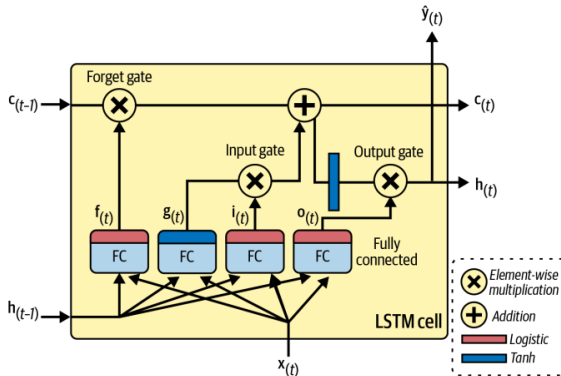


Figure 15-12. An LSTM cell

- Moreover, the result $c_{(t)}$ is also passed through the **tanh** function and then filtered by the **output gate**. This produces the short-term state $h_{(t)}$, which is equal to the LSTM cell's output, $y_{(t)}$, for this time step.

LSTM Cell (cont.)

- For the short-term state $\mathbf{h}_{(t-1)}$, new memories come from the input $\mathbf{x}_{(t)}$. Both $\mathbf{h}_{(t-1)}$ and $\mathbf{x}_{(t)}$ are fed to four different **fully connected (FC)** layers:
 - The main layer is the one that outputs $\mathbf{g}_{(t)}$. It has the usual role of analyzing the current inputs $\mathbf{x}_{(t)}$ and the previous (short-term) state $\mathbf{h}_{(t-1)}$. In an LSTM cell, this layer's output is partially stored in the long-term state.
 - The three other layers are **gate controllers**. Since they use the logistic activation function, their outputs range from 0 to 1.
 - The **forget gate** (controlled by $\mathbf{f}_{(t)}$) controls which parts of the long-term state should be erased.
 - The **input gate** (controlled by $\mathbf{i}_{(t)}$) controls which parts of $\mathbf{g}_{(t)}$ should be added to the long-term state.
 - Finally, the **output gate** (controlled by $\mathbf{o}_{(t)}$) controls which parts of the long-term state should be read and output at this time step (both to $\mathbf{h}_{(t)}$ and $\mathbf{y}_{(t)}$).

LSTM Cell (cont.)

In short, an LSTM cell can learn to recognize an important input (that is the role of the **input gate**), store it in the long-term state, learn to preserve it for as long as it is needed (that is the role of the **forget gate**), and learn to extract it whenever it is needed.

This explains why they have been amazingly successful at capturing long-term patterns in time series, long texts, audio recordings, and more.

LSTM computations:

$$\mathbf{i}_{(t)}^\top = \sigma \left(\mathbf{x}_{(t)}^\top \mathbf{W}_{xi} + \mathbf{h}_{(t-1)}^\top \mathbf{W}_{hi} + \mathbf{b}_i^\top \right)$$

$$\mathbf{f}_{(t)}^\top = \sigma \left(\mathbf{x}_{(t)}^\top \mathbf{W}_{xf} + \mathbf{h}_{(t-1)}^\top \mathbf{W}_{hf} + \mathbf{b}_f^\top \right)$$

$$\mathbf{o}_{(t)}^\top = \sigma \left(\mathbf{x}_{(t)}^\top \mathbf{W}_{xo} + \mathbf{h}_{(t-1)}^\top \mathbf{W}_{ho} + \mathbf{b}_o^\top \right)$$

$$\mathbf{g}_{(t)}^\top = \tanh \left(\mathbf{x}_{(t)}^\top \mathbf{W}_{xg} + \mathbf{h}_{(t-1)}^\top \mathbf{W}_{hg} + \mathbf{b}_g^\top \right)$$

$$\mathbf{c}_{(t)} = \mathbf{f}_{(t)} \otimes \mathbf{c}_{(t-1)} + \mathbf{i}_{(t)} \otimes \mathbf{g}_{(t)}$$

$$\mathbf{y}_{(t)} = \mathbf{h}_{(t)} = \mathbf{o}_{(t)} \otimes \tanh(\mathbf{c}_{(t)})$$

GRU Cells

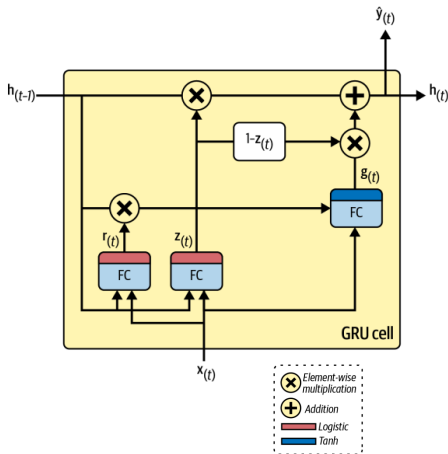


Figure 15-13. GRU cell

The GRU cell is a simplified version of the LSTM cell:

- Both state vectors are merged into a single vector $\mathbf{h}(t)$
- A single gate controller $\mathbf{z}(t)$ controls both the forget gate and the input gate.
- There is no output gate; the full state vector is output at every time step.

GRU Cells (cont.)

GRU computations:

$$\mathbf{z}_{(t)}^\top = \sigma \left(\mathbf{x}_{(t)}^\top \mathbf{W}_{xz} + \mathbf{h}_{(t-1)}^\top \mathbf{W}_{hz} + \mathbf{b}_z^\top \right)$$

$$\mathbf{r}_{(t)}^\top = \sigma \left(\mathbf{x}_{(t)}^\top \mathbf{W}_{xr} + \mathbf{h}_{(t-1)}^\top \mathbf{W}_{hr} + \mathbf{b}_r^\top \right)$$

$$\mathbf{g}_{(t)}^\top = \tanh \left(\mathbf{x}_{(t)}^\top \mathbf{W}_{xg} + \left(\mathbf{r}_{(t)}^\top \mathbf{W}_{hg} \otimes \mathbf{h}_{(t-1)}^\top \right) \right) + \mathbf{b}_g^\top$$

$$\mathbf{h}_{(t)} = \mathbf{z}_{(t)} \otimes \mathbf{h}_{(t-1)} + (1 - \mathbf{z}_{(t)}) \otimes \mathbf{g}_{(t)}$$

Example:

```
# Number of trainable parameters: 1401
gru_model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(shape=[None, 1]),
    tf.keras.layers.GRU(20),
    tf.keras.layers.Dense(1)
])
```

Validation MAE from `gru_model` = 26,497.

Using 1D Convolution Layers to Process Sequences

Similar to using 2D convolution layers for images, we can use 1D convolutional layer for sequences to produce 1D feature map per kernel. If you use 10 kernels, then the layer's output will be composed of 10 1D sequences (all of the same length), or equivalently you can view this output as a single 10D sequence.

```
# Number of trainable parameters: 1283
cnn_gru_model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(shape=[None, 1]),
    tf.keras.layers.Conv1D(filters=8, kernel_size=3, strides=1, activation="relu"),
    tf.keras.layers.Conv1D(filters=16, kernel_size=3, strides=2, activation="relu"),
    tf.keras.layers.GRU(10),
    tf.keras.layers.Dense(1)
])
```

If you train and evaluate this model, you will find that it performed similarly well compared to those previous models. Its validation MAE was 30,209.

It is actually possible to use only 1D convolutional layers and drop the recurrent layers entirely!

Exercises: The `lstm_model`, `gru_model`, and `cnn_gru_model` above use only the #rail riders as input and predict the #rail riders on the 57th day. Modify the codes to take in 5-feature input and train the model.

Summary

- Know when it would be suitable to use RNNs
- Understand recurrent neurons and memory cells
- Understand input and output sequences in a RNN
- Know how to implement a simple RNN in TensorFlow
- Know how to handle variable length input and output sequences
- Know how to train a simple RNN to predict time series
- Know how to build a deep RNN
- Understand the difficulty of training over many time steps
- Understand the LSTM and GRU architectures
- Know how to use 1D convolutional layers to process sequences

For the next topic...

Read up to Chapters 17 on **Autoencoders, GANs, and Diffusion Models** (up to page 647)

And that's all for this lecture.

Have a good week.